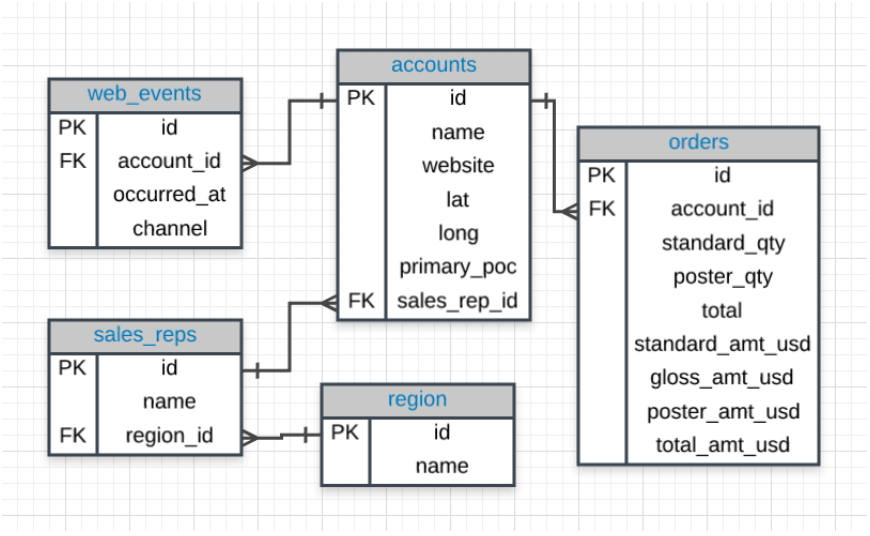
SQL Learning 2 - Udacity Course - SQL BASICS

"C:\Program Files\PostgreSQL\18\bin\postgres.exe" -D "C:\Program Files\PostgreSQL\18\data"

1 #Entity relationship diagramme

ERD : Entity relationship diagramme,is a way to view data in a database,below is the ERD of the db we will use in this course, the ERD helps you to see how the data are stored,tables names,colomnes names,and the relationship between the tables.



2 #How DataBases store data

data are stored as tables which are spreadsheets like excel, every table has a colonnes and rows,

for each colonne all the rows needs to be filled with the same data type

3 #Types of statments

we have three types of statments for now.

1. **CREATE TABLE** is a statement that creates a new table in a database.

2. **DROP TABLE** is a statement that removes a table in a database.
3. **SELECT** allows you to read data and display it. This is called a **query**.

The **SELECT** statement is the common statement used by analysts, and you will be learning all about them throughout this course!

4 #SELET... FROM...

for every query you will use select..from..

1. SELECT: indicates the culomns you want to get
2. FROM: specifies which tables you want to select from,make sure the columns exists in the table

in Order to select everyting from a table we do as follows .

```
select * from table-name
```

every query you gonna use will have select from clauses such as the key word select is used for selectiong the columns you want and the from key word to select the tables you want to retreive the data from.

5 #Formating your queries

writing your queries in upper case or lower case does not affect anything while the logical aspect is respected , but is better to capitlize sql key words and keep the columns and tables in upper case to notice the difference between these two while coding, it keeps reading queries simpler

in postgresql , using a space in variables name need to be inside double quotes to avoid any parsing problems

6 #Key word : LIMIT

1 #DEF:

the key word LIMIT is used to see just few rows of your table, is faster to load than loading all the dataset

the LIMIT key word is always found at the last of the query

```
ex :
```

```
select * from orders LIMIT 10
```

7 #Key word : ORDER BY

1 #DEF:

ORDER BY statement is used to sort the results using any data column, but if your run another query after order by it will affect the unsorted data again.

this statement comes after SELECT..FROM

we can use the key words **ASC** ,**DESC** to ajuste our sorting in the order by key word

```
SELECT id, occurred_at, total_amt_usd
FROM orders
ORDER BY occurred_at
LIMIT 10;
```

we can sort by using more than one column , just you need to separate the values by commas

the sorting occurs from left to right

```
select id,account_id, total_amt_usd from orders
order by account_id asc,total_amt_usd desc
```

8 #Key word : WHERE

1.DEF:

with the WHERE statement we can display subsets of the data on a condition, we can say it is used to filtering the data

Common symbols used in **WHERE** statements include:

1. `>` (greater than)
2. `<` (less than)
3. `>=` (greater than or equal to)
4. `<=` (less than or equal to)
5. `=` (equal to)
6. `!=` (not equal to)

we put it after the from statement so before the order by

```
select *
from orders
where gloss_amt_usd >=1000
limit 5
```

2.WHERE with non numeric data:

we can use the symbols with the non numerical values as well , just make sure to put the value of the condition between single quotes

in common while working with non numerical data in WHERE we use **LIKE**, IN,NOT operators

```
select name,website,primary_poc from accounts where name='E
xxon Mobil'
```

9 #Arithmitics operators :

1.Derived column:

is a column that we add to manipulate data in it , we can use an alias to name it as we want, this last does not affect the database or anything , it is our column we use to to operations on our data, it is a temporary one

familiar mathematical operators will be useful:

1. `*` (Multiplication)
2. `+` (Addition)

3. `-` (Subtraction)
4. `/` (Division)

```
SELECT id, (standard_amt_usd/total_amt_usd)*100 AS std_perc
ent, total_amt_usd
FROM orders
LIMIT 10;
```

to create an **ALIAS** we use the keyword **AS (as we see in the code above)**

10 #Logical operators :

1. **LIKE** :This allows you to perform operations similar to using **WHERE** and `=`, but for cases when you might **not** know **exactly** what you are looking for.
2. **IN** :This allows you to perform operations similar to using **WHERE** and `=`, but for more than one condition.
3. **NOT** : This is used with **IN** and **LIKE** to select all of the rows **NOT LIKE** or **NOT IN** a certain condition.
4. **AND & BETWEEN** : These allow you to combine operations where all combined conditions must be true.
5. **OR** : This allows you to combine operations where at least one of the combined conditions must be true.

10 #LIKE :

we use the LIKE statement in a WHERE clause to deal with text,this operator is usually used with `%` , this symbol tells that we want any number of letters after it , like a search, remember to put the text to look for between single quotes

```
#find all the accounts starting with the letter C
select * from accounts WHERE name LIKE 'C%'
#find all the accounts with the word one somewhere in the n
ame
select name from accounts where name like '%one%'
```

```
#find qll the accounts witname ending in S
select name from account where
```

11 #IN OPERATOR :

IN operator is used to work with both numerical and non numerical data , it allows you to use `=` for more than one value , for numerical data we use single quotes as follows

```
select * from account where name in ('value', 'value2')
select * from web_events where channel IN ('organic','adwor
ds');
# select all the informations of the indiviuals that wre co
ntacted by organis and adwords channel
```

12 #NOT OPERATOR :

it shows the inverse result of IN operator

```
NOT IN #the usage of NOT operator
```

it can be used with both IN and LIKE operators as follows

`NOT IN` `NOT LIKE` we can pull all the rows that does not match the criteria

```
select * from web_events where channel not in ('organic','a
dwords')
select * from accounts where name not like 'C%'
```

13 # AND , Between OPERATOR :

AND Operator

this operator is used in a where statement to consider more than one logical clause

```
select * from orders where standard_qty > 1000 AND poster_q
ty = 0 AND gloss_qty = 0
select * from accounts where name not like 'C%' and name no
t like '%s'
```

BETWEEN Operator

Sometimes we can make a cleaner statement using **BETWEEN** than we can using **AND**. Particularly this is true when we are using the same column for different parts of our **AND** statement. In the previous video, we probably should have used **BETWEEN**.

```
SELECT occurred_at, gloss_qty
FROM orders
WHERE gloss_qty BETWEEN 24 AND 29;
```

14 # OR, OPERATOR :

such as AND operator, `OR` , is used to combine multiple statements, while using this operator we may need to use parentheses to make sure that our logic is performed correctly

```
select * from orders where standard_qty = 0 AND (gloss_qty
> 1000 or poster_qty > 1000)
```

15 # RECAPE :

Statement	How to use it	Other details	
SELECT	`SELECT Col1, Col2, ...`	Choose the columns to return.	
FROM	`FROM Table`	Specify the table containing the columns.	
LIMIT	`LIMIT 10`	Limit the number of rows returned.	

Statement	How to use it	Other details	
ORDER BY	`ORDER BY Col DESC`	Sort the results by a column; use `DESC` for descending order.	
WHERE	`WHERE Col > 5`	Filter results using a condition.	
LIKE	`WHERE Col LIKE '%me%'`	Match text containing a pattern.	
IN	`WHERE Col IN ('Y', 'N')`	Keep rows whose column matches one of the listed values.	
NOT	`WHERE Col NOT IN ('Y', 'N')`	Negate a condition; often used with `IN` or `LIKE`.	
AND	`WHERE Col1 > 5 AND Col2 < 3`	Keep rows only when all conditions are true.	
OR	`WHERE Col1 > 5 OR Col2 < 3`	Keep rows when at least one condition is true.	
BETWEEN	`WHERE Col BETWEEN 3 AND 5`	Select values in a range; often shorter than combining conditions with `AND`.	

SQL Learning 2 - Udacity Course - SQL JOINS

SQL Learning 2 - Udacity Course - SQL AGGREGATIONS